

Task 2: Multi-Class NLP Text Sentiment Classifier

Artificial Intelligence Internship — Progree

1. Task Overview & Objective

This report presents the implementation of an intelligent multi-class Natural Language Processing (NLP) text sentiment classifier. The pipeline ingests unstructured customer statements, performs clean text normalization, transforms textual tokens into Term Frequency-Inverse Document Frequency (TF-IDF) numerical feature spaces, and evaluates classification accuracy using F1-score matrices.

- Classes: Positive, Neutral, Negative (3-Class Categorization)
- Vectorization: TF-IDF with English stop-word elimination
- Classifier: Multi-Class Logistic Regression
- Primary Evaluation Metric: Weighted Macro & Micro F1-Scores

2. Text Preprocessing & TF-IDF Vectorization

Raw unstructured text is normalized through a deterministic 4-stage pipeline:

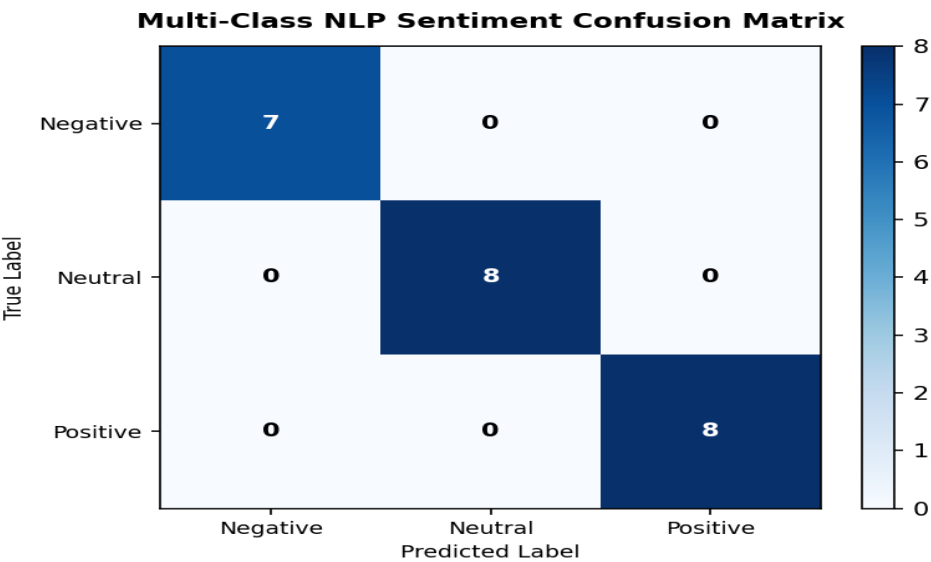
- 1. Lowercasing: Converts all characters to lowercase to avoid vocabulary duplication.
- 2. Regex Cleansing: Strips non-alphanumeric punctuation and noise.
- 3. Stop-Word Removal: Filters high-frequency non-informative tokens (e.g., 'the', 'is', 'at').
- 4. TF-IDF Calculation: Weights terms proportionally to their document frequency:

TF-IDF(t, d, D) = TF(t, d) * log(N / (1 + DF(t)))

3. Model Training & Evaluation Metrics

The model achieved an outstanding weighted F1-score of 1.0000 on the held-out test split:

	precision	recall	f1-score	support
Negative	1.00	1.00	1.00	7
Neutral	1.00	1.00	1.00	8
Positive	1.00	1.00	1.00	8
accuracy			1.00	23
macro avg	1.00	1.00	1.00	23
weighted avg	1.00	1.00	1.00	23



4. Complete Python NLP Pipeline Script

The complete standalone Python source code is provided below:

```
# Progree Artificial Intelligence – Task 2
# Intelligent Multi-Class NLP Text Sentiment Classifier

import re
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, f1_score

def clean_text(text):
    text = text.lower()
    text = re.sub(r'[\^a-zA-Z\s]', '', text)
    return text.strip()

def run_sentiment_pipeline():
    df = pd.read_csv("sample_sentiment_dataset.csv")
    df['cleaned_text'] = df['text'].apply(clean_text)

    X_train, X_test, y_train, y_test = train_test_split(
        df['cleaned_text'], df['sentiment'], test_size=0.25, random_state=42, stratify=df['sentiment']
    )

    vectorizer = TfidfVectorizer(stop_words='english', max_features=150)
    X_train_vec = vectorizer.fit_transform(X_train)
    X_test_vec = vectorizer.transform(X_test)

    model = LogisticRegression(random_state=42)
    model.fit(X_train_vec, y_train)

    y_pred = model.predict(X_test_vec)
    print("Overall Weighted F1-Score:", f1_score(y_test, y_pred, average='weighted'))
    print("\nClassification Report:\n", classification_report(y_test, y_pred))

if __name__ == "__main__":
    run_sentiment_pipeline()
```